

Federated Unlearning under Attack: Poisoning Threats and Defensive Strategies

Wenbin Wang*, Qiwen Ma*, Zifan Zhang, Yuchen Liu, Zhuqing Liu, and Minghong Fang

Abstract—Federated learning facilitates collaborative training of a global model by multiple clients with the support of a central server. However, its decentralized nature exposes it to the risk of poisoning attacks, where malicious clients send harmful local updates to the server, corrupting the global model. To address this, federated unlearning has been developed to eliminate the influence of malicious clients and restore an accurate global model. Despite its significance, most research on federated unlearning has focused on improving its performance and efficiency, neglecting the critical security aspects involved.

In this work, we fill this gap by introducing BadUnlearn, a novel poisoning attack specifically designed to disrupt federated unlearning. In BadUnlearn, malicious clients send intentionally crafted local updates during the unlearning phase with the goal of ensuring the model remains poisoned, even after unlearning efforts. To counter this threat, we propose UnlearnGuard, a resilient federated unlearning framework that offers provable protection against both conventional poisoning attacks and BadUnlearn. The central concept of UnlearnGuard involves the server estimating the local model updates from clients during unlearning and applying a filtering strategy to validate the accuracy of these estimations. Our theoretical analysis demonstrates that the model produced by UnlearnGuard closely approximates one obtained through training from scratch. Through empirical evaluation, we show that BadUnlearn can successfully undermine current federated unlearning techniques, while UnlearnGuard maintains robust security and defense against poisoning attacks, providing a secure solution to the challenges faced in federated unlearning.

Index Terms—Federated unlearning, Poisoning attacks, Robustness.

I. INTRODUCTION

Since the groundbreaking work by McMahan et al. [42], federated learning (FL) has emerged as a powerful distributed learning paradigm. It allows a large number of clients (such as edge devices) to collaboratively train a model under the coordination of a central server, all without sharing their raw training data. Owing to its commitment to respecting clients' privacy, FL has found widespread application in various fields [2],

[3], [51]. The decentralized framework of FL, while offering significant advantages in terms of privacy and efficiency, also introduces inherent vulnerabilities, particularly to poisoning attacks [19], [67], [53], [13], [71], [18], [36].

Numerous Byzantine-robust aggregation techniques have been developed to protect FL systems from poisoning attacks [67], [14], [20], [12], [46], [58], [64], [47], [48], [8], [22], [50], [70], [43], [21], [69]. These methods focus on minimizing the effects of poisoning attacks by either addressing them on a dimension-by-dimension basis or by considering entire vectors. However, recent research has demonstrated that these Byzantine-robust aggregation protocols remain susceptible to sophisticated poisoning attacks. As a result, researchers have introduced various detection-based methods to identify malicious clients within the FL system. For instance, FLDetector [70] identifies malicious clients by assessing the consistency of their local model updates. Despite their potential, these detection-based methods encounter significant limitations: they cannot entirely eliminate the influence of malicious clients, even if all such clients are successfully detected. Specifically, by the time these methods effectively identify malicious clients, the global model may have already been poisoned.

To eliminate the impact of malicious clients on the final learned global model, one obvious approach is to retrain the model entirely from scratch, a process often referred to as *train-from-scratch*. In this approach, the system first identifies and removes the detected malicious clients from the system, and then proceeds to retrain a new global model with the remaining, presumably benign clients. While this method is straightforward, it is highly inefficient, as it introduces significant computational overhead and requires considerable communication resources. Recognizing these inefficiencies, the research community has turned its attention to developing federated unlearning (FU) mechanisms [56], [15], [39], [30], [41]. These mechanisms are designed to unlearn an accurate global model from the corrupted one once malicious clients have been detected. Numerous studies have focused on improving the utility of the FU process, such as its effectiveness and efficiency. However, the security concerns associated with FU have been largely neglected.

Our work: In this work, we bridge this gap by introducing BadUnlearn, a novel poisoning attack specifically designed to target FU. Note that in current FU methods, the server requests clients to calculate exact local model updates and transmit these updates back during specific training rounds to achieve a more accurate global model. This communication

Wenbin Wang is with the School of Information Science and Technology, ShanghaiTech University, China. (e-mail: wangwb2023@shanghaitech.edu.cn).

Qiwen Ma is with the School of Telecommunications Engineering, Xidian University, China. (e-mail: 22012100043@stu.xidian.edu.cn).

Zifan Zhang is with the Department of Computer Science, North Carolina State University, USA. (e-mail: zzhang66@ncsu.edu).

Yuchen Liu is with the Department of Computer Science, North Carolina State University, USA. (e-mail: yuchen.liu@ncsu.edu).

Zhuqing Liu is with the Department of Computer Science and Engineering, University of North Texas, USA. (e-mail: zhuqing.liu@unt.edu).

Minghong Fang is with the Department of Computer Science and Engineering, University of Louisville, USA. (e-mail: minghong.fang@louisville.edu).

*Equal contribution.

†This work is an extension of [60].

between the server and clients during the FU process creates an opportunity for attackers to exploit. In our proposed approach, BadUnlearn, malicious clients send meticulously crafted harmful local model updates to the server throughout the FU process. This manipulation aims to ensure that the final *unlearned model* remains similar to the poisoned model (referred to as the *learned model*) obtained during the FL phase. In other words, BadUnlearn seeks to cause existing FU methods to fail by preventing them from successfully unlearning an accurate global model from the poisoned one. We frame our attack as an optimization problem, which involves determining the local model updates submitted by malicious clients to the server during the FU process. However, solving this optimization problem is challenge due to existing FU methods, like FedRecover [15], where the server employs robust aggregation rules, such as Median [67], to merge client model updates. These robust aggregation rules are generally non-differentiable. To tackle this issue, we propose that each malicious local update is a perturbed form of the learned model.

To mitigate the effects of our proposed BadUnlearn attack, we introduce a provably secure FU framework known as UnlearnGuard. The core principle of UnlearnGuard is that the server can maintain historical information from the system, encompassing clients' local model updates and global models, during the FL process. This historical data provides important insights about the system. Subsequently, in the FU phase, the server can utilize this stored historical information to estimate clients' local model updates. However, the estimated local model updates from clients may not always be precise. To obtain a more accurate unlearned model, we additionally propose to use a filtering strategy to verify the accuracy of the estimations. The fundamental idea of our filtering strategy is that an estimated local model update during the FU process should not significantly deviate from the corresponding stored client's local model update from the FL phase. Building on this intuition, we propose the first variant of our robust FU method, called UnlearnGuard-Dist. In UnlearnGuard-Dist, an estimated local model update is considered precise if it is close in distance to the stored client's local model update from the FL phase. However, UnlearnGuard-Dist does not account for the direction of the estimated clients' model updates. This oversight means that an attacker could still compromise the UnlearnGuard-Dist method by submitting model updates with smaller magnitudes but entirely arbitrary directions. To address this limitation, we introduce another variant of UnlearnGuard, which we call UnlearnGuard-Dir. In UnlearnGuard-Dir, an estimated local model update is deemed precise if it closely matches the direction of its corresponding FL phase update.

Empirically, we evaluate our BadUnlearn attack and UnlearnGuard unlearning method on four real-world datasets (MNIST [35], Fashion-MNIST [63], Human activity recognition [5], and Purchase [1]), 6 existing poisoning attacks (Trim attack [19], Backdoor attack [6], Min-Max attack [53], Min-Sum attack [53], A little is enough attack [7], and Adaptive attack [53]), 8 aggregation rules (FedAvg [42], Median [67], Trimmed-mean [67], FoolsGold [23], Bulyan [26], FABA [62],

Krum [8], and FLAME [46]), and 5 unlearning methods (Train-from-scratch, Historical-information-only [15], FedRecover [15], FATS [56], and TFUA [54]). We demonstrate that current unlearning methods either fail to unlearn an accurate global model even in non-adversarial scenarios or remain susceptible to our proposed BadUnlearn attack. Additionally, our proposed UnlearnGuard can reliably achieve an accurate unlearned model under various attack scenarios, including strong Adaptive attack. Theoretically, we proof that the model unlearned using UnlearnGuard closely approximates the one achieved through train-from-scratch. We summarize our main contributions in this paper as follows:

- We present BadUnlearn attack, a novel poisoning attack targeting the FU process.
- We introduce UnlearnGuard, a FU framework that withstands both existing poisoning attacks and our BadUnlearn. We theoretically demonstrate that, under mild assumptions, the model unlearned by UnlearnGuard closely approximates the one trained from scratch.
- Extensive experiments across multiple benchmark datasets, poisoning attacks, aggregation rules, and FU methods show that BadUnlearn effectively compromises existing FU methods, while our proposed UnlearnGuard remains robust.

II. PRELIMINARIES AND RELATED WORK

A. Federated Learning (FL)

Consider n clients who aim to collaboratively train a machine learning model. Typically, this is done by merging their individual datasets $\{D_1, D_2, \dots, D_n\}$ into one unified dataset $D = D_1 \cup D_2 \cup \dots \cup D_n$, which is subsequently used to train a common model. Here, D_i represents the local training data of client i , where $i \in [n]$. Specifically, in FL, the n clients work together to train a global machine learning model, $\mathbf{w}$, by solving the following finite-sum minimization problem: $\min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}, D) = \sum_{i=1}^n \frac{|D_i|}{|D|} \mathcal{L}_i(\mathbf{w}, D_i)$, where d is the dimension of the model $\mathbf{w}$, $\mathcal{L}_i(\mathbf{w}, D_i)$ is client i 's loss objective, $|D_i|$ is the size of its local dataset, and $|D| = \sum_{i=1}^n |D_i|$ is the total number of examples across all clients. To tackle this optimization problem without centralizing the data, the clients iteratively train the global model across a total of T global training rounds. Specifically, during the t -th round of training, FL executes the following three steps:

- **Step I (Global model broadcasting):** The server distributes the current global model $\mathbf{w}^t$ to the participating clients, which may include all clients or just a selected subset.
- **Step II (Local model training):** Each client i trains its local model using the global model $\mathbf{w}^t$ and computes its update as $\mathbf{g}_i^t = \frac{1}{|B_i^t|} \sum_{k \in B_i^t} \nabla \mathcal{L}_i(\mathbf{w}^t, D_{i,k})$. Here, $D_{i,k} \sim D_i$ represents the k -th training example in the mini-batch B_i^t at client i . Client i then transmits $\mathbf{g}_i^t$ back to the server.
- **Step III (Global model updating):** Once the server receives the model updates from the clients, it applies a specific aggregation rule, ARR, to combine these updates

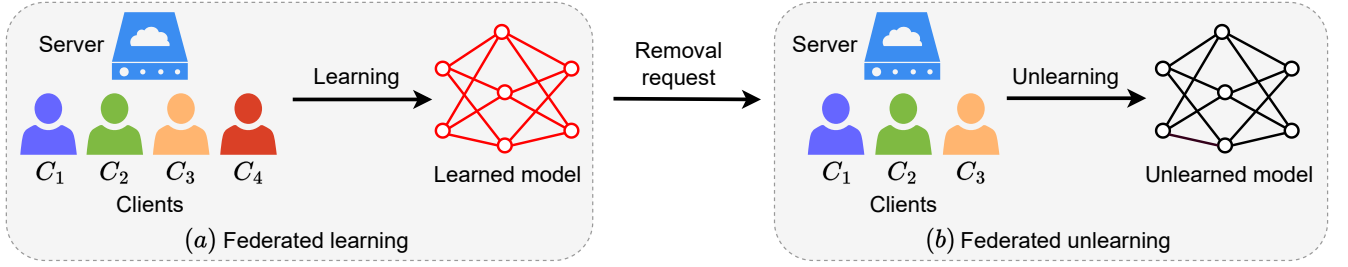


Fig. 1: Illustration of federated learning (FL) and federated unlearning (FU) processes.

into a global model update, which is then used to update the global model accordingly:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \cdot \text{ARR}\{\mathbf{g}_i^t : i \in [n]\}, \quad (1)$$

where η is the learning rate.

B. Poisoning Attacks to FL

FL is widely recognized as being susceptible to poisoning attacks due to its distributed structure [19], [67], [53], [13], [71], [18], [36]. In particular, an attacker can disrupt the training process of an FL system by gaining control over certain malicious clients. These could either be fake clients injected by the attacker or benign clients that have been compromised. The compromised clients can then transmit carefully manipulated local model updates to the server to achieve the goal of the attacker. Poisoning attacks in FL can be categorized based on the attacker's objectives into *untargeted* and *targeted* attacks. In untargeted attacks [19], [67], [67], [53], the attacker's goal is to cause the final global model to make incorrect predictions across a broad range of testing examples without discrimination. In a targeted attack [6], the attacker manipulates the FL system so that the final global model predicts the attacker-selected testing examples as the attacker-chosen labels. Meanwhile, the global model continues to perform well on other testing examples.

C. Byzantine-robust Aggregation Rules

In FL, the most basic and direct method for the server to combine the local model updates from clients is by averaging them. However, as demonstrated in [8], this averaging method is inherently susceptible to poisoning attacks. To enhance the robustness of FL, a variety of Byzantine-robust aggregation techniques have been developed to counteract such attacks [67], [14], [12], [46], [58], [64], [48], [50], [70], [43], [49], [29]. For example, the Median [67] serves as a coordinate-wise Byzantine-robust aggregation method. When the server receives local model updates from clients, it computes the median of these updates for each dimension to produce the final aggregated result. In addition to coordinate-wise robust aggregation methods, vector-wise Byzantine-robust aggregation mechanisms [8], [12] have been introduced to mitigate the effects of poisoning attacks. Moreover, in FLDetector [70], the server seeks to detect the malicious clients.

TABLE I: Summary of full-knowledge, partial-knowledge, and black-box attacks.

Attack type	ARR	Benign updates	Malicious updates
Full-knowledge attack	✓	✓	✓
Partial-knowledge attack	✓	✗	✓
Black-box attack	✗	✗	✓

D. Federated Unlearning (FU)

Machine unlearning [61], [9], [45], [33], [52], [17], [44], [16], [27], [66], [31], [40], [24], [32], [59], [38] is a growing area of research focused on developing methods to erase specific data points from a model after training. As an example, Deltagrad [61] utilizes stored data from the model's initial training process to efficiently remove specific training data from a machine learning model. Recently, there has been growing interest in the study of federated unlearning (FU) [56], [15], [39], [30], [41], [28], [57], [68], [25], [55], which aims to mitigate the influence of malicious clients after the global model has been trained. Unlike machine unlearning, which eliminates specific training data, FU is concerned with removing entire clients from the system. For example, the work in [15] adapted the Deltagrad method to the FL environment, showing its effectiveness in this decentralized context. However, existing research on FU has primarily focused on enhancing the utility of FU algorithms, emphasizing performance and accuracy, while overlooking critical security concerns.

III. THREAT MODEL

Attacker's goal and knowledge: We consider a scenario where an attacker controls some malicious clients. During the FU process, these clients send crafted model updates to ensure the unlearned model remains similar to the model learned from the FL process. As shown in Fig. 1, during FL (Fig. 1(a)), clients C_1 – C_4 participate, with a malicious client like C_4 sending harmful updates that compromise the global model ("Learned model"). If the server later detects C_4 as malicious, it applies an unlearning method (e.g., FedRecover [15]) to restore an accurate model ("Unlearned model" in Fig. 1(b)). However, in our attack, another malicious client like C_3 , stay dormant or inactive during FL, sends crafted updates during FU, disrupting the unlearning process. This ensures the unlearned model remains similar to the compromised one, leaving the "Unlearned model" still poisoned after FU.

Based on the attacker's level of knowledge, we categorize our attacks into three types: *full-knowledge*, *partial-knowledge*,

and *black-box* attacks. In a full-knowledge attack, the attacker can eavesdrop on communications between clients and the server during FL, gaining access to other clients' local updates. Additionally, the attacker can intercept information during FU and is aware of the server's aggregation rule. This type of attack is widely used in existing studies [15] to assess the robustness of FU methods. In a partial-knowledge attack, the attacker has access only to the updates of malicious clients and the aggregation rule ARR. In contrast, a black-box attack limits the attacker's knowledge to the updates of malicious clients, without any information about benign clients' updates or the server's aggregation rule. Table I provides a summary of full-knowledge, partial-knowledge, and black-box attacks.

Our attack differs from traditional FL poisoning attacks, which aim to corrupt training and produce a biased or inaccurate model. Instead, we target the FU process to ensure the unlearned model closely matches the learned one. For instance, if an attacker poisons the FL process, leading to a model with high testing error, our attack ensures the unlearned model retains this high error, making unlearning ineffective. Similarly, if no FL attack occurs and the learned model achieves low testing error, our attack ensures the unlearned model maintains this low error.

It is important to emphasize that this attack is highly feasible in practice. An attacker can control several malicious clients, with some launching attacks during the FL process and subsequently being removed from the system. Meanwhile, **other malicious clients can stay dormant, behaving like benign ones (i.e., not engaging in any attacks during FL) throughout the FL process.** These dormant clients can then activate and carry out attacks during the FU process, ultimately resulting in a compromised unlearned model.

Defender's goal: We aim to develop an effective unlearning method that satisfies the following two objectives: I) Robustness: The unlearning method should be resilient against various poisoning attacks during the FU process, enabling the server to obtain an accurate unlearned model. In other words, the unlearned model produced by our method should be as accurate as a model train-from-scratch, which is the ideal outcome for any unlearning method. II) Aggregation rule independence: The unlearning method should be compatible with any aggregation rule the server uses to combine clients' local model updates. Note that the server is unaware of the attacks initiated by the attacker and has no information about the number of malicious clients involved during the FU process.

IV. OUR BadUnlearn

In existing FU methods such as FedRecover [15], the server requests clients to compute and send their local model updates back during specific rounds, and then aggregates these updates using an aggregation rule, ARR. This communication between clients and the server creates an opportunity for malicious clients to compromise the FU process by submitting harmful model updates. Specifically, in our proposed BadUnlearn attack, malicious clients strategically craft and transmit local model updates to the server in certain rounds during the FU

process, with the goal of ensuring that the resulting unlearned model remains close to the final learned model obtained in the FL phase. Let $\#w$ denote this learned model (e.g., $\#w$ is the "Learned model" shown in Fig. 1(a)). This attack can then be formulated as the following optimization problem:

$$\min \|\#w - \text{ARR}\{g_i^t : i \in [n]\}\|, \quad (2)$$

where $\text{ARR}\{g_i^t : i \in [n]\}$ denotes the aggregated model update after the attack during the FU process, $\|\cdot\|$ denotes the ℓ_2 norm.

However, solving the above Problem (2) directly is difficult because most aggregation rules are not differentiable. To overcome this difficulty, inspired by [53], we employ several practical techniques to approximately solve the problem. We assume that each model update from a malicious client is a perturbed version of the aggregated model update prior to the attack. Let $\mathcal{B}$ represent the set of malicious clients; then, for each malicious client $k \in \mathcal{B}$, we have the following:

$$g_k^t = \#w + \epsilon\psi, \quad k \in \mathcal{B}, \quad (3)$$

where ϵ is a scaling factor, and ψ is a perturbation vector. Furthermore, to simplify solving the optimization problem, we convert Eq. (2) into $\max_{\epsilon, \psi} -\|\#w - \text{ARR}\{g_i^t : i \in [n]\}\|$, as maximizing $-f(x)$ is equivalent to minimizing $f(x)$ when $f(x)$ is the objective function of the optimization problem. Thus, we can reframe the optimization problem in the following form:

$$\begin{aligned} \arg \max_{\epsilon, \psi} & -\|\#w - \text{ARR}\{g_i^t : i \in [n]\}\| \\ \text{s.t.} & \quad g_k^t = \#w + \epsilon\psi, \quad k \in \mathcal{B}. \end{aligned} \quad (4)$$

Solving Problem (4) is challenging due to the simultaneous optimization of ϵ and ψ . To simplify, we fix ψ as a predefined parameter, such as $\psi = -\text{sign}(\#w)$, where sign denotes the coordinate-wise sign of $\#w$. Thus, we reformulate our optimization problem as:

$$\begin{aligned} \arg \max_{\epsilon} & -\|\#w - \text{ARR}\{g_i^t : i \in [n]\}\| \\ \text{s.t.} & \quad g_k^t = \#w + \epsilon\psi, \quad k \in \mathcal{B}, \\ & \quad \psi = -\text{sign}(\#w). \end{aligned} \quad (5)$$

After transforming our optimization into Problem (5), the key challenge boils down to how to efficiently compute the scaling factor ϵ . There are various methods to determine ϵ ; in our paper, we employ the following heuristic approach to obtain ϵ . In each round, if the value of $-\|\#w - \text{ARR}\{g_i^t : i \in [n]\}\|$ increases, then increment ϵ by $\epsilon = \epsilon + \epsilon_{\text{step}}$; if the value decreases, then decrement ϵ by $\epsilon = \epsilon - \epsilon_{\text{step}}$, where ϵ_{step} is the step size for ϵ . Once ϵ is determined, the attacker generates each malicious client's update using Eq. (3) and sends these crafted updates to the server to disrupt the unlearning process.

V. UnlearnGuard

A. Overview

In our proposed UnlearnGuard, the server retains historical information, such as clients' local updates and global models, throughout the FL process. After removing clients responsible for poisoning attacks during the FL process, the server uses the FU technique to unlearn and rebuild a clean global model.

It initializes a new model and trains it iteratively, following the three steps in Section II-A. However, in Step II, instead of requiring clients to compute exact updates, the server estimates them using stored historical information. If these estimates prove inaccurate, the server verifies them and, if necessary, requests the client to compute the exact update.

B. Model Updates Prediction

In our UnlearnGuard, the server leverages historical data from the FL process to estimate client updates during FU. Let $\tilde{\mathbf{g}}_i^t$ be the local update of client i at round t during FL, saved by the server. $\mathbf{g}_i^t$ denotes the exact update during FU, and $\bar{\mathbf{g}}_i^t$ is its estimate. Similarly, $\tilde{\mathbf{w}}^t$ and $\mathbf{w}^t$ represent the global models at round t during FL and FU, respectively. The estimated update $\bar{\mathbf{g}}_i^t$ can be computed using the Cauchy mean value theorem [34] as:

$$\bar{\mathbf{g}}_i^t = \tilde{\mathbf{g}}_i^t + \mathbf{H}_i^t(\mathbf{w}^t - \tilde{\mathbf{w}}^t), \quad (6)$$

where $\mathbf{H}_i^t$ denotes the Hessian matrix for the i -th client at the training round t , and is calculated as $\mathbf{H}_i^t = \int_0^1 \mathbf{H}(\tilde{\mathbf{w}}^t + y(\mathbf{w}^t - \tilde{\mathbf{w}}^t)) dy$.

From Eq. (6), we can see that during the FU process, rather than calculating the clients' exact local model updates, the server can estimate these updates using the information stored during the FL process along with the current global model at round t in the FU process. However, directly computing the estimated model update $\bar{\mathbf{g}}_i^t$ based on Eq. (6) is computationally expensive, as it requires the calculation of the integrated Hessian matrix $\mathbf{H}_i^t$. In the following, we demonstrate how to approximate the integrated Hessian matrix using the standard L-BFGS algorithm [10], [11]. Instead of directly calculating the Hessian, the L-BFGS algorithm approximates the Hessian matrix using a limited history of information from previous training rounds. Let $\Delta \mathbf{w}^t = \mathbf{w}^t - \tilde{\mathbf{w}}^t$ represent the global model difference at training round t , and $\Delta \mathbf{g}_i^t = \bar{\mathbf{g}}_i^t - \tilde{\mathbf{g}}_i^t$ represent the local model update difference for client i at training round t . Additionally, let $\Delta \mathbf{W}^{t,s} = \{\Delta \mathbf{w}^{t-s}, \Delta \mathbf{w}^{t-s+1}, \dots, \Delta \mathbf{w}^{t-1}\}$ denote the global model differences from the previous s rounds, and $\Delta \mathbf{G}_i^{t,s} = \{\Delta \mathbf{g}_i^{t-s}, \Delta \mathbf{g}_i^{t-s+1}, \dots, \Delta \mathbf{g}_i^{t-1}\}$ represent the local model update differences for client i over the last s rounds. The L-BFGS method uses $\Delta \mathbf{W}^{t,s}$, $\Delta \mathbf{G}_i^{t,s}$, and $\mathbf{w}^t - \tilde{\mathbf{w}}^t$ as inputs to compute the Hessian-vector products $\mathbf{H}_i^t(\mathbf{w}^t - \tilde{\mathbf{w}}^t)$. These Hessian-vector products are then used to calculate the predicted local model update $\bar{\mathbf{g}}_i^t$.

C. UnlearnGuard-Dist

In Section V-B, we show that the server can estimate client updates during FU using stored historical information, but inaccuracies may arise, affecting the global model. Over time, these errors can accumulate, reducing model quality. We next propose a calibration technique to minimize these errors.

The first method we propose is UnlearnGuard-Dist, a distance-based calibration technique designed to minimize estimation errors. The central idea of UnlearnGuard-Dist is that, during a specific training round, if a client's local model update estimated in the FU process significantly deviates from

the corresponding local model update stored during the FL process, it indicates an inaccurate estimation. Specifically, we consider the estimated update $\bar{\mathbf{g}}_i^t$ accurate if it satisfies the following condition:

$$\max_{t_1 \in [t-r, t]} \|\bar{\mathbf{g}}_i^t - \tilde{\mathbf{g}}_i^{t_1}\| \leq \max_{t_2, t_3 \in [t-r, t]} \|\tilde{\mathbf{g}}_i^{t_2} - \tilde{\mathbf{g}}_i^{t_3}\|, \quad (7)$$

where r is a buffer parameter that defines the range of recent training rounds we consider (from $t-r$ to t). The left side of the inequality measures the largest distance between the estimated update $\bar{\mathbf{g}}_i^t$ and any of the past r stored updates $\tilde{\mathbf{g}}_i^{t_1}$. The right side measures the largest distance between any two of these past stored updates. The rationale behind this condition is to ensure that the estimated update $\bar{\mathbf{g}}_i^t$ aligns with the client's recent behavior. If the estimated update does not deviate more from the historical updates than those updates deviate among themselves, it suggests that $\bar{\mathbf{g}}_i^t$ is within the normal variation of the client's updates. This consistency indicates that the estimation is reliable. By enforcing this condition, we ensure that the server only uses estimated updates that are plausible given the client's update history. When the condition of Eq. (7) is satisfied, the server uses the estimated update $\bar{\mathbf{g}}_i^t$ to compute the unlearned model at training round t . If the condition is not met, the server will request client i to compute the exact local model update $\mathbf{g}_i^t$. The server then uses this exact update to compute the unlearned model for that round.

D. UnlearnGuard-Dir

Our proposed UnlearnGuard-Dist uses a distance-based calibration to verify the accuracy of estimated client updates, ensuring they stay within a set threshold. However, UnlearnGuard-Dist only considers the magnitude of updates, ignoring their direction. Thus, even if an estimated update $\bar{\mathbf{g}}_i^t$ meets the distance constraint in Eq. (7), it might misalign with the stored update $\tilde{\mathbf{g}}_i^t$, resulting in suboptimal unlearning. To address this issue, we introduce UnlearnGuard-Dir, a direction-based calibration method. This new approach refines the estimation process by considering both the direction and magnitude of updates, thereby reducing the likelihood of estimation errors during the FU process. In UnlearnGuard-Dir, an estimated update $\bar{\mathbf{g}}_i^t$ is deemed accurate if it meets the following criterion:

$$\max_{t_1 \in [t-r, t]} \cos(\bar{\mathbf{g}}_i^t, \tilde{\mathbf{g}}_i^{t_1}) \leq \max_{t_2, t_3 \in [t-r, t]} \cos(\tilde{\mathbf{g}}_i^{t_2}, \tilde{\mathbf{g}}_i^{t_3}), \quad (8)$$

where $[t-r, t]$ is the range of past r training rounds, and $\cos(\cdot, \cdot)$ measures the cosine similarity between updates. The motivation behind this condition is to ensure that the estimated update $\bar{\mathbf{g}}_i^t$ closely follows the directional trend of the client's historical updates. Specifically, we want $\bar{\mathbf{g}}_i^t$ to be no more aligned with any single past update than the maximum alignment observed between pairs of stored historical updates. This prevents the estimated update from deviating too far in any one direction. If the estimated update $\bar{\mathbf{g}}_i^t$ satisfies Eq. (8), we then adjust its magnitude to ensure it falls within a normal range by applying the following rescaling $\bar{\mathbf{g}}_i^t \leftarrow \frac{\bar{\mathbf{g}}_i^t}{\|\bar{\mathbf{g}}_i^t\|} \times \text{Median}\{\|\tilde{\mathbf{g}}_i^{t-r}\|, \|\tilde{\mathbf{g}}_i^{t-r+1}\|, \dots, \|\tilde{\mathbf{g}}_i^t\|\}$, where $\text{Median}\{\|\tilde{\mathbf{g}}_i^{t-r}\|, \|\tilde{\mathbf{g}}_i^{t-r+1}\|, \dots, \|\tilde{\mathbf{g}}_i^t\|\}$ represents the

Algorithm 1 The UnlearnGuard method.

Input: Set of clients in the FU phase $\mathcal{S} = \{i \mid m+1 \leq i \leq n\}$; global models and model updates information in FL phase $\tilde{\mathbf{w}}^0, \tilde{\mathbf{w}}^1, \dots, \tilde{\mathbf{w}}^T; \tilde{\mathbf{g}}_i^0, \tilde{\mathbf{g}}_i^1, \dots, \tilde{\mathbf{g}}_i^{T-1}$; learning rate η ; L-BFGS buffer size s ; aggregation rule ARR.

Output: Unlearned model $\mathbf{w}^T$.

- 1: Initialize $\mathbf{w}_0 \leftarrow \tilde{\mathbf{w}}^0$.
- 2: **for** $t = 0$ to $T - 1$ **do**
- 3: **if** $t < r$ **then**
- 4: The server broadcasts $\mathbf{w}_t$ to the clients in $\mathcal{S}$.
- 5: // Each client trains its local model and computes model update
- 6: Each client i computes $\mathbf{g}_i^t = \nabla \mathcal{L}_i(\mathbf{w}^t)$.
- 7: // Aggregation and global model updating.
- 8: $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \cdot \text{ARR}(\mathbf{g}_{m+1}^t, \dots, \mathbf{g}_n^t)$.
- 9: **else**
- 10: Update L-BFGS buffers $\Delta \mathbf{W}^{t,s}, \Delta \mathbf{G}_i^{t,s}$.
- 11: **for** $i = m+1$ to n **do**
- 12: $\Delta \mathbf{w}^t = \mathbf{w}^t - \tilde{\mathbf{w}}^t$
- 13: // We denote $\text{diag}(\mathbf{X})$ as the diagonal matrix of $\mathbf{X}$, and $\text{tril}(\mathbf{X})$ the lower triangular matrix of $\mathbf{X}$.
- 14: $\mathbf{A}_i^{t,s} = (\Delta \mathbf{W}^{t,s})^\top \Delta \mathbf{G}_i^{t,s}, \mathbf{D}_i^{t,s} = \text{diag}(\mathbf{A}_i^{t,s})$
- 15: $\mathbf{L}_i^{t,s} = \text{tril}(\mathbf{A}_i^{t,s})$
- 16: $\sigma = ((\Delta \mathbf{g}_i^{t-2})^\top \Delta \mathbf{w}^{t-2}) / ((\Delta \mathbf{w}^{t-2})^\top \Delta \mathbf{w}^{t-2})$
- 17: $\mathbf{P}_1 = \begin{bmatrix} -\mathbf{D}_i^{t,s} & (\mathbf{L}_i^{t,s})^\top \\ \mathbf{L}_i^{t,s} & \sigma (\Delta \mathbf{W}^{t,s})^\top \Delta \mathbf{W}^{t,s} \end{bmatrix}^{-1}$
- 18: $\mathbf{P}_2 = \begin{bmatrix} (\Delta \mathbf{G}_i^{t,s})^\top \Delta \mathbf{w}^t \\ \sigma (\Delta \mathbf{W}^{t,s})^\top \Delta \mathbf{w}^t \end{bmatrix}$
- 19: $\mathbf{p} = \mathbf{P}_1 \mathbf{P}_2$
- 20: $\mathbf{H}_i^t \Delta \mathbf{w}^t = \sigma \Delta \mathbf{w}^t - [\Delta \mathbf{G}_i^{t,s} \quad \sigma (\Delta \mathbf{W}^{t,s})] \mathbf{p}$
- 21: $\tilde{\mathbf{g}}_i^t = \tilde{\mathbf{g}}_i^t + \mathbf{H}_i^t \Delta \mathbf{w}^t$
- 22: // Exact training
- 23: **if** condition in Eq. (7) or Eq. (8) is not met **then**
- 24: Compute $\mathbf{g}_i^t = \nabla \mathcal{L}_i(\mathbf{w}^t)$, and update $\tilde{\mathbf{g}}_i^t \leftarrow \mathbf{g}_i^t$ (rescale $\tilde{\mathbf{g}}_i^t$ if necessary).
- 25: **end if**
- 26: **end for**
- 27: $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \cdot \text{ARR}(\tilde{\mathbf{g}}_{m+1}^t, \dots, \tilde{\mathbf{g}}_n^t)$
- 28: **end if**
- 29: **end for**

median value of the magnitudes of the past r updates, starting from round $t-r$ to the most recent round t . Algorithm 1 shows the pseudocode of our proposed UnlearnGuard algorithm. In Algorithm 1, we assume n total clients, with the first m being malicious and performing attacks during FL. After detection and removal, $n-m$ clients remain for the FU phase.

Remarks on BadUnlearn and UnlearnGuard: In Section IV, we highlight that our proposed BadUnlearn assumes the attacker has knowledge of the clients' exact updates during the FU process. In contrast, the server in UnlearnGuard estimates these updates. To carry out an attack, the attacker can leverage the clients' model updates from the FL process to approximate the exact updates. In BadUnlearn, a malicious client can only attack during communication with the server.

During FU, if the server's estimate of a malicious client's update is accurate, no further training is requested, preventing the attack. If the estimate is inaccurate, the server requests precise training, allowing the malicious client to send harmful updates. Furthermore, our UnlearnGuard method aims to unlearn an accurate global model by removing the influence of malicious clients after they are detected at the end of the FL process. Various approaches exist for detecting malicious clients [46], [15], [37], UnlearnGuard operates independently of the specific detection method used.

E. Security Analysis

Before presenting the theoretical results, we outline some standard technical assumptions from the literature [67], [56], [15], [61].

Assumption 1. The loss function $\mathcal{L}_i$ for client i is μ -strongly convexity and L -smoothness. Specifically, the following inequalities hold for any vectors $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^d$:

$$\begin{aligned} \mathcal{L}_i(\mathbf{w}_1) &\geq \mathcal{L}_i(\mathbf{w}_2) + \nabla \mathcal{L}_i(\mathbf{w}_2)^\top (\mathbf{w}_1 - \mathbf{w}_2) + \frac{\mu}{2} \|\mathbf{w}_1 - \mathbf{w}_2\|^2, \\ \mathcal{L}_i(\mathbf{w}_1) &\leq \mathcal{L}_i(\mathbf{w}_2) + \nabla \mathcal{L}_i(\mathbf{w}_2)^\top (\mathbf{w}_1 - \mathbf{w}_2) + \frac{L}{2} \|\mathbf{w}_1 - \mathbf{w}_2\|^2. \end{aligned}$$

Assumption 2. In the L-BFGS algorithm, the error associated with approximating a Hessian-vector product is systematically bounded. Formally, we have:

$$\|\mathbf{H}_i^t(\mathbf{w}^t - \tilde{\mathbf{w}}^t) + \tilde{\mathbf{g}}_i^t - \tilde{\mathbf{g}}_i^t\| \leq M, \quad \text{for any } i \text{ and } t,$$

where M is a finite positive value.

Theorem 1. Suppose Assumptions 1 and 2 hold, and the learning rate satisfies $\eta \leq \min(\frac{1}{\mu}, \frac{1}{L})$, coupled with effective detection of all malicious clients in the FL phase. Then the discrepancy between the global model unlearned through UnlearnGuard-Dist (or UnlearnGuard-Dir) and the model obtained by train-from-scratch in each round $t > 0$ can be bounded such as:

$$\begin{aligned} \|\mathbf{w}^t - \tilde{\mathbf{w}}^t\| &\leq (\sqrt{1 - \eta\mu})^t \|\mathbf{w}^0 - \tilde{\mathbf{w}}^0\| + \frac{1 - (\sqrt{1 - \eta\mu})^t}{1 - \sqrt{1 - \eta\mu}} \eta M \\ &\lesssim \mathcal{O}(\|\mathbf{w}^0 - \tilde{\mathbf{w}}^0\|), \end{aligned}$$

where $\mathbf{w}^t$ and $\tilde{\mathbf{w}}^t$ respectively are the global models obtained by UnlearnGuard-Dist (or UnlearnGuard-Dir) and train-from-scratch in round t , $\mathcal{O}$ is used to denote an upper bound in the asymptotic sense and $f(x) \lesssim g(x)$ indicates that there exists a constant C satisfies that $f(x) \leq Cg(x)$.

Proof. The proof is relegated to Section IX in supplemental material. $\square$

VI. EXPERIMENTS

A. Experimental Setup

1) *Datasets:* We evaluate our methods across four datasets for different learning tasks. We utilize two image classification datasets (MNIST [35] and Fashion-MNIST [63]), and human activity recognition (HAR) [5] dataset, and Purchase [1] dataset.

MNIST [35]: The MNIST dataset is used for the task of recognizing handwritten digits. It includes 60,000 training examples and 10,000 testing examples, with each image categorized into one of ten classes.

Fashion-MNIST [63]: Fashion-MNIST dataset has the same image size, classes, and structure of training and testing splits as MNIST. It consists of front-facing thumbnail images of 70,000 unique products, excluding white-colored items.

HAR [5]: The HAR dataset refers to a publicly available dataset for human activity recognition using smartphones. It was created from recordings of 30 participants performing activities of daily living while wearing a waist-mounted smartphone equipped with inertial sensors. This dataset includes a total of 10,299 instances across 6 distinct classes.

Purchase [1]: This data represents the process of providing incentives (such as coupons) to a large customer base and predicting which customers will develop loyalty to the product. The prediction is made based on customers' shopping histories spanning over a year. The Purchase dataset comprises 197,324 examples, divided into 180,000 training examples and 17,324 testing examples, with each example classified into one of the hundred categories.

2) *Non-IID Setting:* In FL systems, training data is typically unevenly distributed among clients, leading to Non-IID (not independently and identically distributed) conditions. To simulate this Non-IID distribution for the MNIST and Fashion-MNIST datasets, we adopted the approach described in [19]. This approach involves categorizing clients into s random groups, each corresponding to one of the s classes in the dataset. Training examples labeled z are assigned to clients in group z with a probability q , while clients in other groups receive these examples with a probability of $\frac{1-q}{s-1}$. Within each group, the training data is evenly distributed among the clients. As the value of q increases, the degree of Non-IID in the data intensifies. In our experiments, we set $q = 0.5$ by default. It is important to note that we do not simulate a Non-IID setting for the HAR and Purchase datasets, as these datasets are naturally heterogeneous.

3) *Aggregation Rules:* During the FL training and unlearn processes, the server applies a specific aggregation rule to combine the local model updates received from clients. In our experiments, we primarily focus on three aggregation rules: FedAvg [42], Median [67], and Trimmed-mean (TrMean) [67].

FedAvg [42]: When the server receives local model updates from each client, it straightforwardly calculates the average of these updates.

Median [67]: This aggregation rule is both coordinate-wise and robust. The server calculates the median of the local model updates from clients for each individual dimension.

Trimmed-mean (TrMean) [67]: In this aggregation approach, the server begins by discarding the top b highest and bottom b lowest values for each dimension, where b is the trim parameter. It then calculates the average of the remaining elements. b is set to the number of malicious clients by default.

Additionally, we consider five other aggregation rules, including FoolsGold [23], Bulyan [26], FABA [62], Krum [8], and FLAME [46] in Table VII.

TABLE II: The CNN architecture.

Layer	Size
Input	$28 \times 28 \times 1$
Convolution + ReLU	$3 \times 3 \times 30$
Max Pooling	2×2
Convolution + ReLU	$3 \times 3 \times 50$
Max Pooling	2×2
Fully Connected + ReLU	100
Softmax	10

4) *Poisoning Attack Schemes:* We consider the following attacks.

Trim attack [19]: This attack is devised to disrupt the FL process. In this attack, the attacker meticulously crafts the local model updates on the malicious clients so that the global model update after-attack are drastically different from the one prior to the attack.

Backdoor attack [6]: This targeted attack involves malicious clients duplicating their data, embedding triggers, assigning them an attacker's label, and training on the altered dataset. Clients then amplify updates and send them to the server. The backdoor triggers are configured the same way as in [15].

BadUnlearn: This is our proposed attack, where the attacker seeks to manipulate the FU process. The attacker meticulously designs the local model updates on malicious clients, following the approach outlined in Section IV.

5) *FL and FU Settings:* For MNIST and Fashion-MNIST datasets, 2,000 training rounds were performed with a learning rate of 3×10^{-4} and a batch size of 32; for Purchase, 1,000 training rounds were conducted with a learning rate of 1×10^{-4} and a batch size of 2,000; for HAR, 1,000 training rounds were carried out with a learning rate of 3×10^{-4} and a batch size of 32. We assume a total of 100 clients for the MNIST, Fashion-MNIST, and Purchase datasets. For the HAR dataset, which naturally involves 30 participants, there are 30 clients. In the FL process, following the approach in [15], we consider 20% of the clients to be malicious. For model training, we use different architectures based on the dataset. A convolutional neural network (CNN) is employed for the MNIST and Fashion-MNIST datasets, the detail of CNN is shown in Table II. For the HAR dataset, we use a logistic regression classifier. Meanwhile, the Purchase dataset is trained using a fully connected neural network with a single hidden layer, consisting of 1,024 neurons and utilizing the Tanh activation function.

We adopt the same settings for FU as in FL, including training rounds, learning rate, batch size, and aggregation rules. For example, if FL uses the Median rule, FU also applies Median [67]. Following [15], malicious clients that carry out attacks during FL are detected and subsequently removed from the system. Therefore, FU involves 80 clients for MNIST, Fashion-MNIST, and Purchase datasets, and 24 for HAR, with 20% of clients launching attacks like Trim attack [19] or our proposed BadUnlearn. The server can still use Byzantine-robust aggregation to detect malicious clients during FU. In UnlearnGuard, the buffer parameter is set to $r = 5$ for all datasets. We assume the full-knowledge attack as the default setting. By default, we assume malicious clients

TABLE III: Results of different aggregation rules under various attacks, where the attacker manipulates the FL process. The results of Backdoor attacks are presented as “TER / ASR”.

Dataset	Attack FL	FedAvg	Median	TrMean
MNIST	No attack	0.04	0.06	0.06
	Trim attack	0.23	0.41	0.20
	Backdoor attack	0.02 / 0.99	0.09 / 0.01	0.06 / 0.01
Fashion-MNIST	No attack	0.21	0.27	0.26
	Trim attack	0.41	0.48	0.47
	Backdoor attack	0.22 / 0.18	0.28 / 0.02	0.25 / 0.01
HAR	No attack	0.02	0.05	0.04
	Trim attack	0.24	0.15	0.30
	Backdoor attack	0.04 / 0.99	0.06 / 0.02	0.05 / 0.08
Purchase	No attack	0.13	0.13	0.10
	Trim attack	0.29	0.23	0.22
	Backdoor attack	0.24 / 0.05	0.16 / 0.01	0.17 / 0.03

perform Trim attack during FL and BadUnlearn attack during FU. The server applies the Median rule in both the FL and FU processes. Results are reported on the MNIST dataset unless stated otherwise.

6) *Evaluation Metrics*: We consider two metrics: testing error rate (TER) and attack success rate (ASR). Higher TER and ASR indicate stronger attacks, lower values reflect better defenses.

Testing error rate (TER): TER represents the proportion of test examples that the global model misclassifies.

Attack success rate (ASR): The ASR is the proportion of targeted test instances that are classified as the label chosen by the attacker.

7) *Compared Unlearning Methods*: By default, we compare our proposed UnlearnGuard with the following unlearning methods.

Train-from-scratch: In this approach, after identifying and excluding malicious clients, the global model is retrained entirely from the scratch. This process begins with random initialization of parameters and follows the standard FL protocol for training.

Historical-information-only (Historical) [15]: In this Historical method, the server rebuilds a global model by exclusively utilizing local model updates stored during the FL process. Rather than training with fresh data, the server begins by initializing a model and then incrementally refines it using the historical local updates collected from the clients. Note that this method does not involve any communication between the server and clients during FU.

FedRecover [15]: FedRecover also seeks to estimate clients’ local model updates, but it accurately computes these updates at various points throughout the FU process. The server must determine the exact local model updates from clients during the initial warm-up phases, at regular correction intervals, during training rounds that fail anomaly detection, and in the final stages of fine-tuning.

Additionally, we evaluate two other FU methods, FATS [56] and TFUA [54]. The results are presented in Table XIV.

B. Experimental Results

Our BadUnlearn can effectively compromise existing FU methods, and our UnlearnGuard is robust: Table III presents

the results of various aggregation strategies (such as FedAvg, Median, and TrMean) under different attacks across four datasets, where the attacker manipulates the FL process. In the case of Backdoor attacks, the results are expressed as “TER / ASR”. It is important to note that “No attack” refers to a scenario where all clients are benign, and no malicious clients are involved. Table IV demonstrates that the FL process has been compromised by an attacker, and the server attempts to obtain a clean global model by employing either the Train-from-scratch or Historical methods. “Attack FL” means malicious clients perform attack during FL process. Note that we do not carry out attacks on the Train-from-scratch and Historical methods, as these methods involve no communication between the clients and the server during FU.

From Table III, we can see that aggregation strategies like FedAvg, Median, and TrMean are susceptible to poisoning attacks in the FL process. For example, the Median and TrMean rules are particularly vulnerable to Trim attacks. On the MNIST dataset, for instance, the TERs for Median and TrMean under a Trim attack are 0.41 and 0.20, respectively, whereas these values are 0.06 when all clients are benign. Table IV shows that the Historical method fails to unlearn a clean global model when the FL process is compromised. For instance, under a Trim attack with Median aggregation, its TER reaches 0.90. In contrast, the Train-from-scratch method can rebuild a clean model but requires costly retraining, making it impractical. However, it serves as a baseline, with an unlearning method deemed effective if its model’s TER and ASR match those of the Train-from-scratch approach.

Table V and Table VI present the results of the FedRecover, UnlearnGuard-Dist, and UnlearnGuard-Dir methods on MNIST and Fashion-MNIST datasets, respectively. The results for the HAR and Purchase datasets are presented in Table XV and Table XVI in the supplemental material. “Attack FU” means malicious clients perform attack during FU. The goal of the server is to unlearn a clean global model from the poisoned one. For example, during the FL process, the attacker launches the Trim attack. When the server attempts to recover a clean global model using the FedRecover method during the FU process, the attacker continues to target the system, compromising the FU process as well. Based on Tables III-VI, and Tables XV-XVI, we can draw several observations.

First, our proposed BadUnlearn effectively compromises the FedRecover method, a leading FU technique. For example, on the MNIST dataset, a Trim attack during FL yields a TER of 0.20 with the TrMean rule (Table III). Even after applying FedRecover, BadUnlearn maintains pressure during FU, resulting in a TER of 0.23, demonstrating its success in disrupting the unlearning process. Second, FedRecover shows robustness against Trim attacks during the FU phase, which aligns with the conclusions drawn in [15].

Third, our proposed FU methods, UnlearnGuard-Dist and UnlearnGuard-Dir, demonstrate robustness against various poisoning attacks and can accurately unlearn a clean global model from a compromised one. Specifically, our UnlearnGuard-Dir method produces a global model with classification accuracy comparable to that of a model obtained through Train-from-scratch. For instance, on the Fashion-

TABLE IV: Results of Train-from-scratch and Historical methods on four datasets, where the attacker manipulates the FL process, and the server aims to obtain a clean global model.

(a) MNIST dataset.

Attack FL	ARR	Train-from-scratch	Historical
Trim attack	FedAvg	0.05	0.90
	Median	0.07	0.90
	TrMean	0.06	0.90
Backdoor attack	FedAvg	0.05 / 0.00	0.85 / 0.00
	Median	0.06 / 0.00	0.89 / 0.00
	TrMean	0.06 / 0.00	0.86 / 0.00

(b) Fashion-MNIST dataset.

Attack FL	ARR	Train-from-scratch	Historical
Trim attack	FedAvg	0.22	0.90
	Median	0.25	0.90
	TrMean	0.24	0.90
Backdoor attack	FedAvg	0.20 / 0.02	0.87 / 0.00
	Median	0.22 / 0.01	0.87 / 0.00
	TrMean	0.21 / 0.02	0.88 / 0.00

(c) HAR dataset.

Attack FL	ARR	Train-from-scratch	Historical
Trim attack	FedAvg	0.05	0.84
	Median	0.06	0.43
	TrMean	0.06	0.75
Backdoor attack	FedAvg	0.05 / 0.00	0.71 / 0.00
	Median	0.07 / 0.00	0.79 / 0.00
	TrMean	0.06 / 0.00	0.83 / 0.00

(d) Purchase dataset.

Attack FL	ARR	Train-from-scratch	Historical
Trim attack	FedAvg	0.13	0.97
	Median	0.13	0.86
	TrMean	0.13	0.93
Backdoor attack	FedAvg	0.11 / 0.02	0.98 / 0.00
	Median	0.12 / 0.00	0.99 / 0.00
	TrMean	0.12 / 0.00	0.99 / 0.00

TABLE V: Results of FedRecover, UnlearnGuard-Dist, and UnlearnGuard-Dir methods on the MNIST dataset, where the attacker manipulates the FL and FU processes.

Attack FL	Attack FU	ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
No attack	No attack	FedAvg	0.06	0.05	0.05
		Median	0.05	0.06	0.04
		TrMean	0.07	0.07	0.04
	Trim attack	FedAvg	0.05	0.06	0.06
		Median	0.12	0.12	0.08
		TrMean	0.15	0.11	0.06
	BadUnlearn	FedAvg	0.06	0.05	0.05
		Median	0.07	0.11	0.07
		TrMean	0.06	0.13	0.05
Trim attack	No attack	FedAvg	0.05	0.06	0.05
		Median	0.11	0.14	0.09
		TrMean	0.15	0.12	0.06
	Trim attack	FedAvg	0.06	0.07	0.06
		Median	0.15	0.14	0.10
		TrMean	0.16	0.15	0.07
	BadUnlearn	FedAvg	0.24	0.07	0.06
		Median	0.39	0.14	0.09
		TrMean	0.23	0.14	0.06
Backdoor attack	No attack	FedAvg	0.05 / 0.00	0.02 / 0.00	0.02 / 0.00
		Median	0.12 / 0.01	0.11 / 0.01	0.10 / 0.00
		TrMean	0.07 / 0.00	0.06 / 0.00	0.06 / 0.00
	Trim attack	FedAvg	0.06 / 0.01	0.03 / 0.01	0.03 / 0.00
		Median	0.12 / 0.00	0.12 / 0.00	0.11 / 0.00
		TrMean	0.04 / 0.00	0.06 / 0.00	0.05 / 0.00
	BadUnlearn	FedAvg	0.04 / 0.98	0.03 / 0.01	0.03 / 0.00
		Median	0.10 / 0.02	0.13 / 0.00	0.12 / 0.00
		TrMean	0.08 / 0.03	0.07 / 0.00	0.07 / 0.00

MNIST dataset, when the attacker manipulates the system during the FL process and the Median rule is used, the TER for Train-from-scratch is 0.25 (see Table IVb). The global model unlearned by our UnlearnGuard-Dir method has a TER of 0.27 (see Table VI) under both the Trim attack and the BadUnlearn attack. Fourth, our proposed UnlearnGuard-Dist and UnlearnGuard-Dir methods are also effective in unlearning clean global models when the attacker targets the system with a Backdoor attack during the FL process.

More complex aggregation rules: This section explores scenarios with advanced aggregation methods, such as FoolsGold [23], Bulyan [26], FABA [62], Krum [8], and FLAME [46]. Note that in our experiments, the same rule is applied across both processes (e.g., if Median is used in

FL, it is also used in FU). The attacker employs a Trim attack in FL and BadUnlearn during FU. The results are shown in Table VII. “Learned model” means the final model at the end of FL process, i.e., $\#w$. As shown in Table VII, our unlearning methods effectively defend against poisoning attacks, even with these sophisticated rules.

More advanced attacks during FL: We also evaluate the scenarios where the clients deploy more sophisticated attacks during FL process, including Min-Max [53], Min-Sum [53], A little is enough (LIE) attack [7]. The results are shown in Table VIII. Note that in Table VIII, malicious clients carry out these advanced attacks during FL, and still perform BadUnlearn during FU. We observe that FedRecover is also vulnerable to our BadUnlearn in this case.

TABLE VI: Results of FedRecover, UnlearnGuard-Dist, and UnlearnGuard-Dir methods on the Fashion-MNIST dataset, where the attacker manipulates the FL and FU processes, and the server aims to obtain a clean model from the poisoned one.

Attack FL	Attack FU	ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
No attack	No attack	FedAvg	0.19	0.16	0.15
		Median	0.22	0.20	0.19
		TrMean	0.23	0.19	0.16
	Trim attack	FedAvg	0.21	0.20	0.17
		Median	0.19	0.18	0.18
		TrMean	0.24	0.21	0.22
	BadUnlearn	FedAvg	0.24	0.19	0.18
		Median	0.28	0.25	0.22
		TrMean	0.28	0.26	0.21
Trim attack	No attack	FedAvg	0.22	0.20	0.20
		Median	0.24	0.30	0.25
		TrMean	0.23	0.28	0.26
	Trim attack	FedAvg	0.23	0.21	0.21
		Median	0.22	0.23	0.19
		TrMean	0.25	0.22	0.21
	BadUnlearn	FedAvg	0.40	0.23	0.21
		Median	0.47	0.31	0.27
		TrMean	0.47	0.30	0.27
Backdoor attack	No attack	FedAvg	0.23 / 0.01	0.21 / 0.00	0.21 / 0.00
		Median	0.26 / 0.04	0.28 / 0.01	0.23 / 0.01
		TrMean	0.27 / 0.02	0.24 / 0.00	0.23 / 0.01
	Trim attack	FedAvg	0.26 / 0.00	0.22 / 0.00	0.21 / 0.00
		Median	0.26 / 0.02	0.28 / 0.01	0.23 / 0.01
		TrMean	0.29 / 0.01	0.25 / 0.00	0.22 / 0.01
	BadUnlearn	FedAvg	0.21 / 0.16	0.24 / 0.00	0.22 / 0.00
		Median	0.31 / 0.03	0.29 / 0.01	0.25 / 0.00
		TrMean	0.27 / 0.02	0.26 / 0.00	0.25 / 0.00

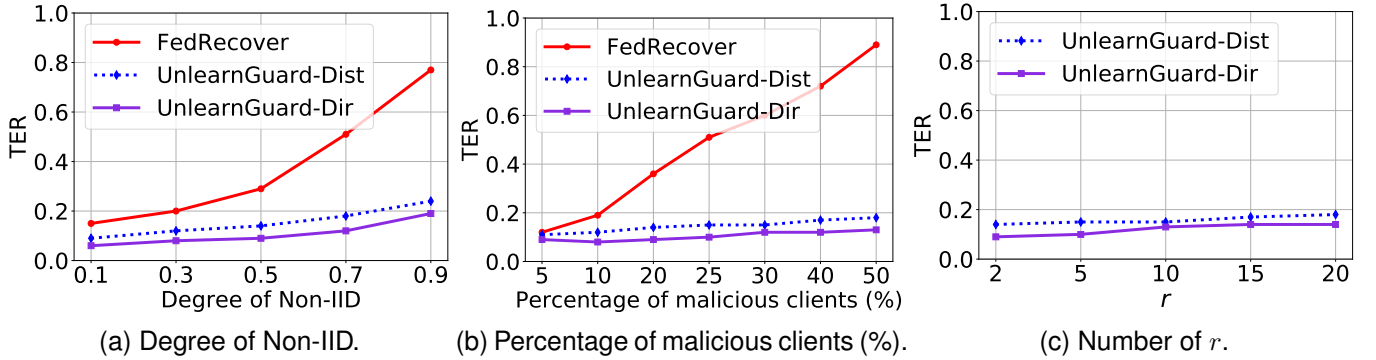


Fig. 2: Impact of degree of Non-IID, percentage of malicious clients during FU, and the number of r .

TABLE VII: The server uses more complex aggregation rules during FL and FU.

ARR	Learned model	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
FoolsGold	0.10	0.17	0.11	0.09
Bulyan	0.13	0.19	0.12	0.13
FABA	0.08	0.19	0.09	0.10
Krum	0.10	0.16	0.11	0.12
FLAME	0.06	0.21	0.09	0.08

TABLE VIII: Malicious clients perform advanced attacks during FL.

ARR	Learned model	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
Min-Max	0.22	0.20	0.13	0.12
Min-Sum	0.23	0.24	0.12	0.09
LIE	0.18	0.19	0.09	0.08

Impact of degree of Non-IID: In this section, we examine how the degree of Non-IID affects unlearning methods.

The results are presented in Fig. 2a. We observe that both UnlearnGuard-Dist and UnlearnGuard-Dir exhibit similar increasing trends, though the TER of UnlearnGuard-Dir remains consistently lower. In contrast, FedRecover has the highest TER, and as the degree of Non-IID increases, the rate at which its TER grows also accelerates. When the degree of Non-IID reaches 0.9, the TER of FedRecover is approximately 0.8, which is four times higher than the TER of UnlearnGuard-Dir.

Impact of fraction of malicious clients during the FU process: In this section, we examine the impact of fraction of malicious clients during the FU process. The fraction of malicious clients in the FL phase is fixed at 20%. During the FU process, we adjust the percentage of malicious clients. The server uses the Median aggregation rule, while the attacker employs the Trim attack in the FL phase and our BadUnlearn during the FU phase. The results are presented in Fig. 2b. We observe that as the percentage of malicious clients increases

TABLE IX: Results of partial-knowledge attack.

ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
FedAvg	0.23	0.07	0.06
Median	0.38	0.14	0.09
TrMean	0.23	0.14	0.06
FoolsGold	0.20	0.08	0.09
Bulyan	0.23	0.13	0.11
FABA	0.24	0.12	0.09
Krum	0.21	0.13	0.13
FLAME	0.24	0.12	0.11

TABLE X: Results of black-box attack. The attacker uses the Median rule.

ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
FedAvg	0.22	0.06	0.05
Median	0.38	0.14	0.08
TrMean	0.23	0.14	0.05
FoolsGold	0.19	0.09	0.09
Bulyan	0.21	0.15	0.12
FABA	0.21	0.12	0.11
Krum	0.18	0.15	0.13
FLAME	0.23	0.14	0.13

from 5% to 50%, the TER of FedRecover also rises, from 0.12 to 0.89. However, our proposed UnlearnGuard-Dist and UnlearnGuard-Dir remain robust against poisoning attacks, even when a large fraction of clients are malicious during the FU process.

Impact of the buffer size r : Fig. 2c presents the impact of buffer size r on the testing error rate (TER) across our proposed recovery algorithms: UnlearnGuard-Dist and UnlearnGuard-Dir. We observe that TER remains generally stable, albeit with a slight increase as r increases. The results highlight the robustness of UnlearnGuard-Dir and UnlearnGuard-Dist, demonstrating their ability to maintain performance even with an increasing number of buffer size r .

Partial-knowledge attack: In this scenario, we examine the partial-knowledge attack, where the attacker lacks information about benign updates but is aware of malicious updates and the aggregation rule employed by the server. Following [53], the attacker generates benign updates using benign data on malicious clients. Under this setting, the attacker executes the Trim attack during FL and deploys our BadUnlearn attack during FU. The results for the partial-knowledge attack are presented in Table IX. As observed in Table IX, our proposed BadUnlearn effectively compromises existing FU approaches, such as FedRecover, even when advanced aggregation rules like FoolsGold, Krum, and FLAME are utilized. In addition, our proposed UnlearnGuard remains robust.

Black-box attack: This section explores the black-box attack, where the attacker possesses knowledge only of updates from malicious clients while remaining unaware of benign updates and the server's aggregation rule. This scenario represents a highly realistic threat model. In our proposed BadUnlearn, we instantiate ARR using the Median rule in Eq. (5). The results of black-box attacks are summarized in Table X, where ARR denotes the aggregation rule actually applied by the server. From Table X, we observe that even without access to benign updates or the server's aggregation strategy, our BadUnlearn successfully disrupts existing FU methods.

TABLE XI: Results of UnlearnGuard under Adaptive attack.

ARR	UnlearnGuard-Dist	UnlearnGuard-Dir
FedAvg	0.06	0.05
Median	0.16	0.08
TrMean	0.15	0.07

TABLE XII: Malicious clients send learned model $\#w$ to the server during FU.

ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
FedAvg	0.07	0.06	0.08
Median	0.14	0.13	0.10
TrMean	0.15	0.15	0.05

VII. DISCUSSION AND LIMITATIONS

Adaptive attack: Here, we evaluate the performance of UnlearnGuard under a more challenging Adaptive attack, where the attacker possesses complete knowledge of the FU process, including all clients' local model updates, the server's aggregation rule, and the server's filtering strategy (e.g., Eq. (7) or Eq. (8)). Moreover, malicious clients execute the Trim attack during the FL phase and the Adaptive attack during FU. The results, presented in Table XI, indicate that both UnlearnGuard-Dist and UnlearnGuard-Dir remain resilient even against such strong attacks.

Malicious clients send learned model $\#w$ to the server during FU: In our threat model, malicious clients intentionally send tailored model updates to the server to ensure that the unlearned model closely resembles the learned model $\#w$ during the FU process. A simple attack strategy involves these malicious clients directly submitting $\#w$ to the server. The results of this attack are presented in Table XII. However, we observe that this approach is ineffective in compromising the FedRecover method.

Remove benign clients after FL process: By default, we assume some malicious clients poison the model during FL, and the server aims to recover a clean model in FU. Here, we explore the scenario with no attacks during FL. Instead of removing 20% of malicious clients, we exclude 20% of benign clients and perform unlearning using the remaining participants. For instance, some benign clients may choose to exit the system after the global model's training is complete, driven by privacy concerns. Among these remaining clients, 20% execute the BadUnlearn attack. The results of this experiment are presented in Table XIII. A comparison between Table III and Table XIII reveals that the FedRecover's performance under the BadUnlearn attack closely matches that of the learned model without any attack, which aligns with our objective defined in Eq. (2).

Attack more complex FU method: In the previous experiments, the server used FedRecover for unlearning, and our BadUnlearn attack successfully compromised it. Here, we explore a scenario where the server employs the advanced FATS [56] or TFUA [54] unlearning method. Malicious clients run BadUnlearn during FU, and results on four datasets in Table XIV show that BadUnlearn effectively disrupts the FATS and TFUA unlearning approaches.

TABLE XIII: Results when removing 20% of benign clients after FL process.

ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
FedAvg	0.05	0.07	0.06
Median	0.07	0.09	0.06
TrMean	0.07	0.11	0.05

TABLE XIV: Results of BadUnlearn attacking the FATS [56] and TFUA [54] unlearning methods.

Method	MNIST	Fashion-MNIST	HAR	Purchase
FATS	0.25	0.42	0.27	0.28
TFUA	0.22	0.30	0.22	0.25

Server storage overhead: In our proposed UnlearnGuard, the server maintains a record of historical information, which allows it to track model updates over time. Specifically, with d parameters per model, the server's storage requirements scale with the number of clients, n , and the total number of training rounds, T , resulting in a storage complexity of $O(dnT)$.

However, this level of storage complexity is quite manageable for powerful servers, particularly those found in modern data centers. These servers are equipped with large-scale storage systems capable of handling such requirements efficiently. In our experiments, the UnlearnGuard implementation required up to 200 GB of storage, which is a substantial but feasible amount. Notably, this storage can be allocated to hard drives, relieving the need to use more expensive and faster main memory, which further demonstrates the practicality and scalability of the approach. Thus, the space complexity of $O(dnT)$ can be effectively managed even in resource-constrained environments, provided the proper infrastructure is in place.

Privacy concern of UnlearnGuard: In our proposed UnlearnGuard, the server retains the model updates from clients during the FL process. However, storing these updates may give rise to privacy concerns, as the model updates could potentially reveal sensitive information about individual clients, such as their local data or specific learning patterns. Nevertheless, these privacy risks can be effectively addressed and mitigated through the adoption of well-established privacy-preserving techniques. For instance, differential privacy [4], [65] can be integrated into our UnlearnGuard to provide a robust layer of protection for client data. By adding carefully calibrated noise to the model updates, differential privacy ensures that no individual client's data can be identified or reconstructed from the aggregated model, even if an attacker has access to the global model and its updates. This approach significantly reduces the likelihood of privacy breaches while still allowing the server to effectively unlearn an accurate global model.

VIII. CONCLUSION

In this paper, we introduced BadUnlearn, a novel poisoning attack specifically designed to compromise the federated unlearning process. By carefully crafting model updates, BadUnlearn prevents the unlearning of malicious clients while evading detection. Through extensive empirical evaluation, we

demonstrated that BadUnlearn significantly degrades the performance of existing federated unlearning systems, exposing vulnerabilities in their ability to handle malicious interference.

To defend against such attacks during the federated unlearning phase, we proposed two robust unlearning methods: UnlearnGuard-Dist and UnlearnGuard-Dir. Both methods were designed to withstand poisoning attacks, ensuring the security of the FU process. We provided both theoretical analysis and empirical evidence showing that UnlearnGuard-Dist and UnlearnGuard-Dir effectively protect against poisoning attacks, making them secure and reliable solutions for secure federated unlearning.

REFERENCES

- [1] *Acquire Valued Shoppers Challenge*.
- [2] Federated learning: Collaborative machine learning without centralized training data. <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>.
- [3] Utilization of fate in risk management of credit in small and micro enterprises. <https://www.fedai.org/cases/utilization-of-fate-in-risk-management-of-credit-in-small-and-micro-enterprises/>.
- [4] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *CCS*, 2016.
- [5] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra Perez, and Jorge Luis Reyes Ortiz. A public domain dataset for human activity recognition using smartphones. In *ESANN*, 2013.
- [6] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *AISTATS*, 2020.
- [7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *NeurIPS*, 2019.
- [8] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *NeurIPS*, 2017.
- [9] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy*, 2021.
- [10] Richard H Byrd, Peihuang Lu, Jorge Nocedal, and Ciyu Zhu. A limited memory algorithm for bound constrained optimization. In *SIAM Journal on scientific computing*, 1995.
- [11] Richard H Byrd, Jorge Nocedal, and Robert B Schnabel. Representations of quasi-newton matrices and their use in limited memory methods. In *Mathematical Programming*, 1994.
- [12] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. Fltrust: Byzantine-robust federated learning via trust bootstrapping. In *NDSS*, 2021.
- [13] Xiaoyu Cao and Neil Zhenqiang Gong. Mpaf: Model poisoning attacks to federated learning based on fake clients. In *CVPR Workshops*, 2022.
- [14] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Provably secure federated learning against malicious clients. In *AAAI*, 2021.
- [15] Xiaoyu Cao, Jinyuan Jia, Zaixi Zhang, and Neil Zhenqiang Gong. Fedrecover: Recovering from poisoning attacks in federated learning using historical information. In *IEEE Symposium on Security and Privacy*, 2023.
- [16] Jiaao Chen and Diyi Yang. Unlearn what you want to forget: Efficient unlearning for llms. *arXiv preprint arXiv:2310.20150*, 2023.
- [17] Min Chen, Weizhuo Gao, Gaoyang Liu, Kai Peng, and Chen Wang. Boundary unlearning: Rapid forgetting of deep networks via shifting the decision boundary. In *CVPR*, 2023.
- [18] Yanbo Dai and Songze Li. Chameleon: Adapting to peer images for planting durable backdoors in federated learning. In *ICML*, 2023.
- [19] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. Local model poisoning attacks to byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.
- [20] Minghong Fang, Seyedsina Nabavirazavi, Zhuqing Liu, Wei Sun, Sundararaja Sitharama Iyengar, and Haibo Yang. Do we really need to design new byzantine-robust aggregation rules? In *NDSS*, 2025.
- [21] Minghong Fang, Zifan Zhang, Hairi, Prashant Khanduri, Jia Liu, Songtao Lu, Yuchen Liu, and Neil Gong. Byzantine-robust decentralized federated learning. In *CCS*, 2024.

- [22] Hossein Fereidooni, Alessandro Pegoraro, Phillip Rieger, Alexandra Dmitrienko, and Ahmad-Reza Sadeghi. Freqfed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning. In *NDSS*, 2024.
- [23] Clement Fung, Chris JM Yoon, and Ivan Beschastnikh. Mitigating sybils in federated learning poisoning. In *arXiv*, 2018.
- [24] Chongyang Gao, Lixu Wang, Chenkai Weng, Xiao Wang, and Qi Zhu. Practical unlearning for large language models. *arXiv preprint arXiv:2407.10223*, 2024.
- [25] Xiangshan Gao, Xingjun Ma, Jingyi Wang, Youcheng Sun, Bo Li, Shouling Ji, Peng Cheng, and Jiming Chen. Verifi: Towards verifiable federated unlearning. In *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [26] Rachid Guerraoui, Sébastien Rouault, et al. The hidden vulnerability of distributed learning in byzantium. In *ICML*, 2018.
- [27] Varun Gupta, Christopher Jung, Seth Neel, Aaron Roth, Saeed Sharifi-Malvajerdi, and Chris Waites. Adaptive machine unlearning. In *NeurIPS*, 2021.
- [28] Anisa Halimi, Swanand Kadhe, Ambrish Rawat, and Nathalie Baracaldo. Federated unlearning: How to efficiently erase a client in fl? *arXiv preprint arXiv:2207.05521*, 2022.
- [29] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *ICML*, 2024.
- [30] Hyejun Jeong, Shiqing Ma, and Amir Houmansadr. Sok: Challenges and opportunities in federated unlearning. *arXiv preprint arXiv:2403.02437*, 2024.
- [31] Jiabao Ji, Yujian Liu, Yang Zhang, Gaowen Liu, Ramana Kompella, Sijia Liu, and Shiyu Chang. Reversing the forget-retain objectives: An efficient llm unlearning framework from logit difference. In *NeurIPS*, 2024.
- [32] Jinghan Jia, Yihua Zhang, Yimeng Zhang, Jiancheng Liu, Bharat Runwal, James Diffenderfer, Bhavya Kailkhura, and Sijia Liu. Soul: Unlocking the power of second-order optimization for llm unlearning. *arXiv preprint arXiv:2404.18239*, 2024.
- [33] Meghdad Kurmanji, Peter Triantafillou, Jamie Hayes, and Eleni Triantafillou. Towards unbounded machine unlearning. In *NeurIPS*, 2024.
- [34] Serge Lang. *Real and functional analysis*, volume 142. Springer Science & Business Media, 2012.
- [35] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010.
- [36] Haoyang Li, Qingqing Ye, Haibo Hu, Jin Li, Leixia Wang, Chengfang Fang, and Jie Shi. 3dfed: Adaptive and extensible framework for covert backdoor attack in federated learning. In *IEEE Symposium on Security and Privacy*, 2023.
- [37] Suyi Li, Yong Cheng, Wei Wang, Yang Liu, and Tianjian Chen. Learning to detect malicious clients for robust federated learning. *arXiv preprint arXiv:2002.00211*, 2020.
- [38] Chris Liu, Yaxuan Wang, Jeffrey Flanigan, and Yang Liu. Large language model unlearning via embedding-corrupted prompts. In *NeurIPS*, 2024.
- [39] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. Federaser: Enabling efficient client-level data removal from federated learning models. In *IWQOS*, 2021.
- [40] Sijia Liu, Yuanshun Yao, Jinghan Jia, Stephen Casper, Nathalie Baracaldo, Peter Hase, Yuguang Yao, Chris Yuhao Liu, Xiaojun Xu, Hang Li, et al. Rethinking machine unlearning for large language models. In *Nature Machine Intelligence*, 2025.
- [41] Ziyao Liu, Yu Jiang, Jiyuan Shen, Minyi Peng, Kwok-Yan Lam, Xingliang Yuan, and Xiaoning Liu. A survey on federated unlearning: Challenges, methods, and future directions. In *ACM Computing Surveys*, 2023.
- [42] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.
- [43] Luis Muñoz-González, Kenneth T Co, and Emil C Lupu. Byzantine-robust federated machine learning through adaptive model averaging. *arXiv preprint arXiv:1909.05125*, 2019.
- [44] Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. Descent-to-delete: Gradient-based methods for machine unlearning. In *Algorithmic Learning Theory*, 2021.
- [45] Thanh Tam Nguyen, Thanh Trung Huynh, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. A survey of machine unlearning. *arXiv preprint arXiv:2209.02299*, 2022.
- [46] Thien Duc Nguyen, Phillip Rieger, Roberta De Viti, Huili Chen, Björn B Brandenburg, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, et al. Flame: Taming backdoors in federated learning. In *USENIX Security Symposium*, 2022.
- [47] Xudong Pan, Mi Zhang, Duocai Wu, Qifan Xiao, Shouling Ji, and Min Yang. Justinian's gaavornor: Robust distributed learning with gradient aggregation agent. In *USENIX Security Symposium*, 2020.
- [48] Jungwuk Park, Dong-Jun Han, Minseok Choi, and Jaekyun Moon. Sage-flow: Robust federated learning against both stragglers and adversaries. In *NeurIPS*, 2021.
- [49] Zeyu Qin, Liuyi Yao, Daoyuan Chen, Yaliang Li, Bolin Ding, and Minhao Cheng. Revisiting personalized federated learning: Robustness against backdoor attacks. In *KDD*, 2023.
- [50] Phillip Rieger, Thien Duc Nguyen, Markus Miettinen, and Ahmad-Reza Sadeghi. Deepsight: Mitigating backdoor attacks in federated learning through deep model inspection. In *NDSS*, 2022.
- [51] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletari, Holger R Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N Galtier, Bennett A Landman, Klaus Maier-Hein, et al. The future of digital health with federated learning. In *NPJ digital medicine*, 2020.
- [52] Ayush Sekhari, Jayadev Acharya, Gautam Kamath, and Ananda Theertha Suresh. Remember what you want to forget: Algorithms for machine unlearning. In *NeurIPS*, 2021.
- [53] Virat Shejwalkar and Amir Houmansadr. Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *NDSS*, 2021.
- [54] Xinyi Sheng, Wei Bao, and Liming Ge. Robust federated unlearning. In *CIKM*, 2024.
- [55] Ningxin Su and Baochun Li. Asynchronous federated unlearning. In *INFOCOM*, 2023.
- [56] Youming Tao, Cheng-Long Wang, Miao Pan, Dongxiao Yu, Xiuzhen Cheng, and Di Wang. Communication efficient and provable federated unlearning. In *VLDB*, 2024.
- [57] Fei Wang, Baochun Li, and Bo Li. Federated unlearning and its privacy threats. *IEEE Network*, 2023.
- [58] Ning Wang, Yang Xiao, Yimin Chen, Yang Hu, Wenjing Lou, and Y Thomas Hou. Flare: defending federated learning against model poisoning attacks via latent space representations. In *ASIACCS*, 2022.
- [59] Qizhou Wang, Jin Peng Zhou, Zhanke Zhou, Saebyeol Shin, Bo Han, and Kilian Q Weinberger. Rethinking llm unlearning objectives: A gradient perspective and go beyond. *arXiv preprint arXiv:2502.19301*, 2025.
- [60] Wenbin Wang, Qiwen Ma, Zifan Zhang, Yuchen Liu, Zhuqing Liu, and Minghong Fang. Poisoning attacks and defenses to federated unlearning. In *The Web Conference*, 2025.
- [61] Yinjun Wu, Edgar Dobriban, and Susan Davidson. Deltagrad: Rapid retraining of machine learning models. In *ICML*, 2020.
- [62] Qi Xia, Zeyi Tao, Zijiang Hao, and Qun Li. Faba: an algorithm for fast aggregation against byzantine attacks in distributed neural networks. In *IJCAI*, 2019.
- [63] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017.
- [64] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *ICML*, 2019.
- [65] Yuchen Yang, Bo Hui, Haolin Yuan, Neil Gong, and Yinzhi Cao. Privateff: Accurate, differentially private federated learning via personalized data transformation. In *USENIX Security Symposium*, 2023.
- [66] Yuanshun Yao, Xiaojun Xu, and Yang Liu. Large language model unlearning. *arXiv preprint arXiv:2310.10683*, 2023.
- [67] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *ICML*, 2018.
- [68] Wei Yuan, Hongzhi Yin, Fangzhao Wu, Shijie Zhang, Tieke He, and Hao Wang. Federated unlearning for on-device recommendation. In *WSDM*, 2023.
- [69] Kaiyuan Zhang, Guanhong Tao, Qiuling Xu, Siyuan Cheng, Shengwei An, Yingqi Liu, Shiwei Feng, Guangyu Shen, Pin-Yu Chen, Shiqing Ma, et al. Flip: A provable defense framework for backdoor mitigation in federated learning. In *ICLR*, 2023.
- [70] Zaixi Zhang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Flde-tector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *KDD*, 2022.
- [71] Zhengming Zhang, Ashwinee Panda, Linyue Song, Yaoqing Yang, Michael Mahoney, Prateek Mittal, Ramchandran Kannan, and Joseph Gonzalez. Neurotoxin: Durable backdoors in federated learning. In *ICML*, 2022.

Supplemental Material to “Federated Unlearning under Attack: Poisoning Threats and Defensive Strategies”

Wenbin Wang, Qiwen Ma, Zifan Zhang, Yuchen Liu, Zhuqing Liu, and Minghong Fang

IX. PROOF OF THEOREM 1

Here, we establish the bound of $\|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|$ between the global model obtained by UnlearnGuard-Dist (or UnlearnGuard-Dir) and the train-from-scratch method. First, we delineate the update rules for unlearning processes.

Estimation phase: we employ both gradient (or local model update) and Hessian information such that:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \sum_{i=m+1}^n \alpha_i [\mathbf{H}_i^t(\mathbf{w}^t - \tilde{\mathbf{w}}^t) + \tilde{\mathbf{g}}_i^t], \quad (9)$$

where $\alpha_i = \frac{|D_i|}{|D'|}$, $D' = \bigcup_{i=m+1}^n D_i$ denotes the joint dataset of the remaining $n - m$ clients. For simplicity, we will adopt these notations in the subsequent analysis.

Exact training phase: we only use the exact gradient of each client such that:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \sum_{i=m+1}^n \alpha_i \mathbf{g}_i^t. \quad (10)$$

Additionally, we denote $\mathbf{h}_i^t$ as the model update contributed by the client i model in the t round of the train-from-scratch strategy. The global model, updated in round t is given by:

$$\ddot{\mathbf{w}}^t = \ddot{\mathbf{w}}^{t-1} - \eta \sum_{i=m+1}^n \alpha_i \mathbf{h}_i^t. \quad (11)$$

Consider **Estimation** phase, where $\mathbf{w}^t$ is updated according to Eq. (9). Based on Eqs. (10) and (11), we have:

$$\begin{aligned} & \|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| \\ &= \|(\mathbf{w}^t - \eta \sum_{i=m+1}^n \alpha_i \mathbf{g}_i^t) - (\ddot{\mathbf{w}}^t - \eta \sum_{i=m+1}^n \alpha_i \mathbf{h}_i^t)\| \\ &= \|\mathbf{w}^t - \ddot{\mathbf{w}}^t - \eta \sum_{i=m+1}^n \alpha_i (\mathbf{g}_i^t - \mathbf{h}_i^t)\|, \end{aligned} \quad (12)$$

Applying triangle inequality, we bound for the difference between $\mathbf{w}^{t+1}$ and $\ddot{\mathbf{w}}^{t+1}$ as:

$$\begin{aligned} & \|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| \\ &\leq \underbrace{\|\mathbf{w}^t - \ddot{\mathbf{w}}^t - \eta \sum_{i=m+1}^n (\alpha_i \|\mathbf{g}_i^t - \mathbf{h}_i^t\|)\|}_{\text{I}} \\ &\quad + \underbrace{\|\eta \sum_{i=m+1}^n \alpha_i [\mathbf{g}_i^t - \mathbf{H}_i^t(\mathbf{w}^t - \tilde{\mathbf{w}}^t) + \tilde{\mathbf{g}}_i^t]\|}_{\text{II}}. \end{aligned} \quad (13)$$

It remains to bound terms **I** and **II**, respectively.

For term **I**, we obtain:

$$\begin{aligned} \text{I}^2 &= \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|^2 - 2\eta \langle \mathbf{w}^t - \ddot{\mathbf{w}}^t, \sum_{i=m+1}^n \alpha_i (\mathbf{g}_i^t - \mathbf{h}_i^t) \rangle \\ &\quad + \eta^2 \left\| \sum_{i=m+1}^n \alpha_i (\mathbf{g}_i^t - \mathbf{h}_i^t) \right\|^2, \end{aligned} \quad (14)$$

where $\langle \cdot, \cdot \rangle$ represents the inner product of two vectors.

By applying the Cauchy-Schwarz inequality, we derive:

$$\begin{aligned} \text{I}^2 &\leq (\|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|^2 - \eta \sum_{i=m+1}^n \alpha_i \langle \mathbf{w}^t - \ddot{\mathbf{w}}^t, \mathbf{g}_i^t - \mathbf{h}_i^t \rangle) \\ &\quad - \eta \sum_{i=m+1}^n \alpha_i \langle \mathbf{w}^t - \ddot{\mathbf{w}}^t, \mathbf{g}_i^t - \mathbf{h}_i^t \rangle + \eta^2 \sum_{i=m+1}^n \alpha_i^2 \|\mathbf{g}_i^t - \mathbf{h}_i^t\|^2. \end{aligned} \quad (15)$$

Given Assumption 1 and Assumption 2, we see the loss function $\mathcal{L}_i$ is μ -strong convexity and L -smoothness. Utilizing inequalities for inner products, we obtain:

$$\begin{aligned} \text{I}^2 &\leq (\|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|^2 - \eta \mu \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|^2) \\ &\quad - \eta \sum_{i=m+1}^n \left(\frac{\alpha_i}{L} - \frac{\eta}{\alpha_i^2} \right) \|\mathbf{g}_i^t - \mathbf{h}_i^t\|^2. \end{aligned} \quad (16)$$

If the learning rate η meets the condition $\eta \leq \frac{1}{L} \leq \frac{|D'|}{L \max_{i=m+1}^n |D_i|}$, we obtain:

$$\text{I}^2 \leq (1 - \eta \mu) \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|^2. \quad (17)$$

Furthermore, we have:

$$\text{I} \leq \sqrt{1 - \eta \mu} \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|. \quad (18)$$

For term **II**, we can derive an upper bound based on Assumption 2 such that:

$$\text{II} \leq \eta M. \quad (19)$$

Substituting (18) and (19) into (13), we have:

$$\|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| \leq \text{I} + \text{II} \leq \sqrt{1 - \eta \mu} \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\| + \eta M. \quad (20)$$

In the following, we consider **Exact training** case, which occurs when $t < r$ and the conditions in Eq. (7) or Eq. (8) are not satisfied. Similar to the previous analysis, we obtain:

$$\|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| = \|\mathbf{w}^t - \ddot{\mathbf{w}}^t - \eta \sum_{i=m+1}^n \alpha_i (\mathbf{g}_i^t - \mathbf{h}_i^t)\|. \quad (21)$$

It is obvious that the RHS of above equation is what we defined earlier as **I**. Hence, we have:

$$\|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| \leq \sqrt{1 - \eta \mu} \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\|. \quad (22)$$

Combining two cases, we have:

$$\|\mathbf{w}^{t+1} - \ddot{\mathbf{w}}^{t+1}\| \leq \sqrt{1 - \eta\mu} \|\mathbf{w}^t - \ddot{\mathbf{w}}^t\| + \eta M. \quad (23)$$

By recursion, we have:

$$\|\mathbf{w}^t - \ddot{\mathbf{w}}^t\| \leq (\sqrt{1 - \eta\mu})^t \|\mathbf{w}^0 - \ddot{\mathbf{w}}^0\| + \frac{1 - (\sqrt{1 - \eta\mu})^t}{1 - \sqrt{1 - \eta\mu}} \eta M, \quad (24)$$

for any $t \geq 0$, where $\mathbf{w}^0$ and $\ddot{\mathbf{w}}^0$ are the initial parameter values in the FU phase and train-from-scratch, respectively. This formulation concludes that under appropriate conditions on η , the bound converges to $\frac{\eta M}{1 - \sqrt{1 - \eta\mu}}$ as $t \rightarrow \infty$, thereby confirming the stability of the learning algorithm.

TABLE XV: Results of FedRecover, UnlearnGuard-Dist, and UnlearnGuard-Dir methods on the HAR dataset, where the attacker manipulates the FL and FU processes, and the server aims to obtain a clean global model from the poisoned one.

Attack FL	Attack FU	ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
No attack	No attack	FedAvg	0.05	0.05	0.04
		Median	0.06	0.06	0.05
		TrMean	0.04	0.05	0.04
	Trim attack	FedAvg	0.06	0.05	0.05
		Median	0.06	0.04	0.05
		TrMean	0.05	0.6	0.05
	BadUnlearn	FedAvg	0.25	0.11	0.10
		Median	0.19	0.13	0.09
		TrMean	0.31	0.12	0.09
Trim attack	No attack	FedAvg	0.05	0.08	0.04
		Median	0.05	0.06	0.06
		TrMean	0.06	0.06	0.06
	Trim attack	FedAvg	0.07	0.08	0.04
		Median	0.06	0.08	0.08
		TrMean	0.07	0.11	0.10
	BadUnlearn	FedAvg	0.26	0.09	0.08
		Median	0.16	0.10	0.09
		TrMean	0.32	0.10	0.07
Backdoor attack	No attack	FedAvg	0.06 / 0.00	0.05 / 0.01	0.05 / 0.00
		Median	0.07 / 0.00	0.06 / 0.00	0.06 / 0.00
		TrMean	0.07 / 0.00	0.06 / 0.00	0.06 / 0.00
	Trim attack	FedAvg	0.07 / 0.01	0.05 / 0.01	0.06 / 0.00
		Median	0.08 / 0.00	0.07 / 0.00	0.05 / 0.00
		TrMean	0.07 / 0.00	0.06 / 0.00	0.06 / 0.00
	BadUnlearn	FedAvg	0.05 / 0.98	0.06 / 0.01	0.07 / 0.00
		Median	0.08 / 0.00	0.07 / 0.00	0.07 / 0.00
		TrMean	0.06 / 0.07	0.06 / 0.00	0.06 / 0.00

TABLE XVI: Results of FedRecover, UnlearnGuard-Dist, and UnlearnGuard-Dir methods on the Purchase dataset, where the attacker manipulates the FL and FU processes, and the server aims to obtain a clean global model from the poisoned one.

Attack FL	Attack FU	ARR	FedRecover	UnlearnGuard-Dist	UnlearnGuard-Dir
No attack	No attack	FedAvg	0.15	0.12	0.11
		Median	0.12	0.11	0.08
		TrMean	0.10	0.09	0.06
	Trim attack	FedAvg	0.19	0.17	0.12
		Median	0.13	0.12	0.11
		TrMean	0.18	0.13	0.10
	BadUnlearn	FedAvg	0.15	0.21	0.13
		Median	0.14	0.14	0.13
		TrMean	0.11	0.13	0.12
Trim attack	No attack	FedAvg	0.25	0.23	0.13
		Median	0.16	0.16	0.13
		TrMean	0.23	0.17	0.13
	Trim attack	FedAvg	0.28	0.26	0.19
		Median	0.18	0.18	0.14
		TrMean	0.27	0.15	0.18
	BadUnlearn	FedAvg	0.30	0.25	0.14
		Median	0.22	0.17	0.16
		TrMean	0.24	0.17	0.17
Backdoor attack	No attack	FedAvg	0.18 / 0.05	0.14 / 0.03	0.13 / 0.01
		Median	0.16 / 0.00	0.15 / 0.00	0.13 / 0.00
		TrMean	0.17 / 0.00	0.14 / 0.00	0.14 / 0.00
	Trim attack	FedAvg	0.20 / 0.03	0.15 / 0.02	0.12 / 0.01
		Median	0.15 / 0.00	0.16 / 0.00	0.13 / 0.00
		TrMean	0.21 / 0.00	0.16 / 0.00	0.12 / 0.00
	BadUnlearn	FedAvg	0.27 / 0.06	0.15 / 0.03	0.15 / 0.01
		Median	0.13 / 0.00	0.16 / 0.00	0.14 / 0.00
		TrMean	0.19 / 0.04	0.16 / 0.00	0.14 / 0.00